

REGIME-ADAPTIVE PORTFOLIO OPTIMIZER

A Quantitative Framework for Regime-Conditioned Allocation and Capacity Stress Testing

Nova Tech Club | Team: Alessandro De Riccardis, Moritz, Max | 2025

1. Project Overview

Most portfolio optimization frameworks assume static market conditions. In practice, markets alternate between clearly distinct regimes: risk-on environments characterized by low volatility and positive momentum; risk-off environments driven by macro stress and flight to safety; and transitional states where signals are mixed. A static optimizer does not adapt to this reality and inevitably misallocates capital at precisely the wrong moments.

This project builds a modular, end-to-end quantitative system that: (1) detects the current market regime using macroeconomic signals; (2) adjusts expected return estimates for a diversified ETF universe accordingly; (3) solves a constrained convex optimization problem to find the optimal portfolio weights; and (4) stress-tests the strategy's capacity and performance degradation under varying AUM, liquidity constraints, and market impact assumptions.

The system is composed of five independent modules, each with a clean interface so that components can be developed in parallel and integrated at a defined checkpoint. The final deliverables are a public GitHub repository, a 10-slide presentation, and a short academic paper (3-4 pages) documenting methodology and results.

WHY IT MATTERS

Institutional portfolio managers face exactly this problem at scale. Understanding how strategy performance degrades with size is as important as the strategy itself. A Sharpe ratio of 1.2 on \$5M becomes meaningless at \$200M if market impact destroys the alpha. This project produces a concrete, quantified answer to that question.

TECH STACK

Python 3.10+ | yfinance | FRED API | hmmlearn | cvxpy | pandas / numpy / matplotlib | GitHub

2. Asset Universe and Data

The portfolio universe consists of 8 liquid ETFs covering major asset classes. This set is intentionally broad and diversified to ensure regime-dependent allocation shifts are meaningful.

Ticker	Asset Class	Role in Portfolio
SPY	US Large Cap Equity	Core risk asset; overweighted in Risk-On regimes
TLT	US Long-Term Treasuries	Safe haven; overweighted in Risk-Off regimes
GLD	Gold	Inflation hedge and crisis buffer
EEM	Emerging Market Equity	High-beta risk asset; reduced in Risk-Off

VNQ	Real Estate (REITs)	Yield-sensitive; moderate allocation
LQD	Investment Grade Credit	Hybrid bond/credit exposure
DBC	Commodities	Macro-sensitive; supports regime signals
IEF	US 7-10 Year Treasuries	Intermediate duration safe haven

Macro features used for regime detection are downloaded from FRED: VIX (CBOE Volatility Index), T10Y2Y (10-year minus 2-year Treasury spread as a yield curve proxy), and NFCI (National Financial Conditions Index). All price data is sourced from Yahoo Finance using the yfinance library. The full historical window is January 2010 to December 2024.

3. Module Architecture

The system is divided into five modules. Each module produces a clearly defined output that serves as the input for the next. Modules 1-3 are sequential dependencies; Module 4 and Module 5 require all prior modules to be complete before integration.

#	Module	Output	Owner
M1	Data Pipeline	Clean DataFrame: prices, returns, macro series	Moritz
M2	Regime Classifier	Time series of regime labels (0/1/2) + current regime	Alessandro
M3	Signal Layer	Vector of expected returns per asset, regime-conditioned	Max
M4	Convex Optimizer	Optimal weight vector w^* per rebalancing period	Moritz
M5	Backtester + Stress Test	Performance metrics, capacity frontier heatmap	Alessandro

4. Module Assignments

MORITZ | Module 1: Data Pipeline | Module 4: Convex Optimizer

Step 1: Data Pipeline

Goal: produce a single, clean master DataFrame that all other modules will import. Nothing else in the project touches raw data directly.

File to create: data_loader.py

Step-by-step instructions:

1. Install dependencies: `pip install yfinance pandas numpy requests fredapi`
2. Define a list of tickers: `['SPY','TLT','GLD','EEM','VNQ','LQD','DBC','IEF']`
3. Use `yfinance.download()` with `auto_adjust=True` to fetch daily Adjusted Close prices from 2010-01-01 to 2024-12-31. Store in a DataFrame called `prices_df`, indexed by date.
4. Compute daily log returns: `returns_df = np.log(prices_df / prices_df.shift(1))`. Drop the first row (NaN).
5. Download macro series from FRED using `fredapi`: VIXCLS (VIX), T10Y2Y (yield curve slope), NFCI (financial conditions). Resample all to business-day frequency using forward-fill (ffill) to handle weekend gaps.
6. Align macro data to the same date index as `prices_df` using `pd.concat` with `join='inner'` to keep only dates present in all series.
7. Check for and handle missing values: assert that no column has more than 5% NaNs; fill remaining gaps with `ffill` then `bfill`.
8. Export two functions: `get_prices()` returns `prices_df`; `get_features()` returns the combined macro feature DataFrame. Both should accept optional start and end date parameters.
9. Write a short test block (if `__name__ == '__main__'`) that calls both functions, prints shapes, and prints the last 3 rows to confirm everything loads correctly.

Expected output: a file `data_loader.py` that any team member can import and call without needing to understand the data sourcing logic.

Step 4: Convex Optimizer

Goal: given a vector of expected returns μ (from Module 3) and a covariance matrix Σ (computed from historical returns), find the optimal portfolio weights w^* that maximize risk-adjusted return subject to real-world constraints.

File to create: `optimizer.py`

Step-by-step instructions:

10. Install `cvxpy`: `pip install cvxpy`
11. Write a function `solve_portfolio(mu, Sigma, lam=2.0, max_weight=0.30, vol_target=0.10, turnover_budget=None, w_prev=None)` that takes expected returns and covariance as inputs and returns the optimal weight vector.
12. Inside the function, define the optimization variable: `w = cp.Variable(n)` where `n` is the number of assets.
13. Define the objective: maximize `w.T @ mu - lam * cp.quad_form(w, Sigma)`. This is the mean-variance objective with risk aversion parameter `lam`.
14. Define the constraints: (a) `cp.sum(w) == 1` (fully invested); (b) `w >= 0` (long only); (c) `w <= max_weight` (maximum 30% per asset); (d) `cp.sqrt(cp.quad_form(w, Sigma) * 252) <= vol_target` (annualized portfolio volatility cap).
15. If `turnover_budget` is not `None` and `w_prev` is not `None`, add the constraint: `cp.norm(w - w_prev, 1) <= turnover_budget`. This is the capacity constraint.
16. Solve with `cp.Problem(cp.Maximize(objective), constraints).solve(solver=cp.SCS)`. Check that the status is 'optimal' before returning `w.value`.
17. Add error handling: if the solver does not return 'optimal', log a warning and return equal weights (`np.ones(n)/n`) as fallback.
18. Write a test block that generates random `mu` and `Sigma` (using `np.random`), calls `solve_portfolio`, and prints the resulting weights and their sum (should equal 1.0).

Key integration note: the function signature must stay fixed. Alessandro and Max will call `solve_portfolio()` from their modules. Do not change parameter names without notifying the team.

MAX | Module 3: Signal Layer | Visualizations | Presentation

Step 3: Signal Layer

Goal: produce a vector of expected returns μ (one value per asset) for each rebalancing date, conditioned on the current regime label provided by Module 2.

File to create: `signals.py`

Step-by-step instructions:

19. Import `returns_df` from `data_loader.py` and the regime series from `regime_classifier.py` (Module 2 output).
20. Compute the momentum signal for each asset: 12-1 month cross-sectional momentum. Formula: for each date t , compute the cumulative return from $t-252$ trading days to $t-21$ trading days. This is the standard momentum definition that excludes the most recent month to avoid short-term reversal noise.
21. Standardize the momentum signal cross-sectionally at each date: subtract the mean and divide by the standard deviation across all 8 assets. Store as a z-score.
22. Define expected returns as a regime-scaled version of the z-score. Create a dictionary `regime_scaling = {0: 1.0, 1: 0.3, 2: 0.6}` where 0=Risk-On, 1=Risk-Off, 2=Transitional. Multiply the z-score by the corresponding scaling factor.
23. Additionally, in Risk-Off regime (label=1), apply an explicit tilt: add +0.10 to TLT and GLD expected return, and subtract -0.10 from EEM and SPY expected return. This represents defensive repositioning.
24. Write a function `get_expected_returns(date, regime_label)` that returns a numpy array of expected returns for the 8 assets at the given date and regime.
25. Test the function on three specific dates: one in each regime. Print the output and verify that the ordering of expected returns makes economic sense (TLT and GLD should rank high in Risk-Off).

Step 3b: Visualizations

Goal: produce all charts used in the final presentation and paper. This is not optional -- clear visualizations are 50% of how the project is communicated.

Charts to produce (save each as a high-resolution PNG at 300 DPI):

- `regime_timeline.png`: Plot the SPY price series from 2010 to 2024 as a line chart. Add colored background bands corresponding to the three regimes (use a transparent fill: green for Risk-On, red for Risk-Off, yellow for Transitional). Label the axes and add a legend. This is the single most important chart in the deck.
- `strategy_comparison.png`: Plot three cumulative return curves on the same chart: Equal Weight (benchmark), Mean-Variance Unconditional, and Regime-Adaptive Optimizer. Use distinct colors. Add a table below the chart with Sharpe ratio, max drawdown, and Calmar ratio for each strategy.

- `capacity_frontier.png`: Heatmap (seaborn.heatmap) with AUM on the x-axis (log scale: \$1M to \$500M) and monthly turnover budget on the y-axis (5% to unlimited). Cell color = Sharpe ratio at that (AUM, turnover) combination. This is the most original output of the entire project.
- `weight_evolution.png`: Stacked area chart showing how portfolio weights evolve over time, colored by asset. Shows visually that the optimizer changes allocation across regimes.

For the presentation (10 slides), follow this structure: Slide 1 - title; Slide 2 - problem statement (why static allocation fails); Slide 3 - system architecture diagram; Slide 4 - regime timeline chart; Slide 5 - the optimizer (explain constraints in plain language); Slide 6 - strategy comparison chart; Slide 7 - capacity frontier heatmap; Slide 8 - key findings; Slide 9 - limitations; Slide 10 - next steps.

ALESSANDRO | Module 2: Regime Classifier | Module 5: Backtester + Stress Test

Step 2: Regime Classifier

Goal: train a Hidden Markov Model (HMM) on macroeconomic features to produce a discrete regime label for each trading day. The output is the central input for all downstream modules.

File to create: `regime_classifier.py`

Step-by-step instructions:

26. Install hmmlearn: `pip install hmmlearn`
27. Import macro features from `data_loader.get_features()`. Use three features: VIX (level), T10Y2Y (level), and a rolling 21-day return of SPY (to capture equity momentum within the macro context).
28. Standardize all three features using `sklearn.preprocessing.StandardScaler`. Fit the scaler only on the training set (2010-2018). Apply the same fitted scaler to the full sample (2010-2024). This is critical -- do not fit on the full sample as it would introduce lookahead bias.
29. Train a Gaussian HMM with 3 hidden states using `GaussianHMM(n_components=3, covariance_type='full', n_iter=100)`. Fit on the training sample only (2010-2018).
30. Decode the full sample using `model.predict(X_scaled)`. This produces a sequence of state labels (0, 1, or 2) for every date from 2010 to 2024.
31. Interpret the states economically: compute the average VIX level for each state. Assign the label 'Risk-On' to the state with the lowest average VIX, 'Risk-Off' to the state with the highest average VIX, and 'Transitional' to the middle state. Remap the integer labels accordingly (0=Risk-On, 1=Risk-Off, 2=Transitional).
32. Validate the regime assignment: print a summary table showing, for each regime, the average VIX, average T10Y2Y, average SPY monthly return, and number of days. The economic interpretation must hold: Risk-Off should have the highest VIX and lowest average SPY return.
33. Export the regime series as a pandas Series with `DatetimeIndex`, and export a function `get_regime(date)` that returns the integer label for any given date.
34. Save the fitted HMM model using `joblib.dump` so it does not need to be retrained at each run.

Important methodological note: the HMM is fitted once on 2010-2018. The backtest in Module 5 uses the model to predict regimes on unseen data (2019-2024). Never refit the model on post-2018 data -- this is the core discipline that makes the results credible.

Step 5: Backtester and Stress Test

Goal: implement a walk-forward backtest to measure real out-of-sample performance, then systematically stress-test the strategy under varying capacity constraints to produce the capacity frontier.

File to create: backtester.py and stress_test.py

Part A -- Walk-Forward Backtester:

- 35. Define the backtest period: 2019-01-01 to 2024-12-31 (out-of-sample only). The period 2010-2018 is used only for training.
- 36. Set rebalancing frequency to monthly. On the last business day of each month, compute new optimal weights and hold them until the next rebalancing date.
- 37. For each rebalancing date t: (a) call get_regime(t) to get the current regime; (b) call get_expected_returns(t, regime) to get mu; (c) compute the rolling 252-day covariance matrix Sigma from returns_df up to t; (d) call solve_portfolio(mu, Sigma, w_prev=w_t-1) to get w*; (e) apply proportional transaction costs of 5bps per unit of turnover: cost = 0.0005 * sum(abs(w* - w_prev)).
- 38. Compute daily portfolio returns as the dot product of daily asset returns and the current weight vector.
- 39. Run the same backtest for three strategies in parallel: (a) Equal Weight -- fixed 1/N allocation, rebalanced monthly; (b) Mean-Variance Unconditional -- optimizer with no regime conditioning (flat mu for all regimes); (c) Regime-Adaptive -- the full system.
- 40. Compute final metrics for each strategy: annualized Sharpe ratio (using 252 trading days), maximum drawdown, Calmar ratio (annualized return / max drawdown), annualized turnover, and total transaction costs.
- 41. Export results as a summary DataFrame and save daily return series for all three strategies for use in visualization.

Part B -- Capacity Stress Test:

- 42. Define a grid: AUM_grid = [1e6, 5e6, 10e6, 50e6, 100e6, 200e6, 500e6] (in USD); turnover_grid = [0.05, 0.10, 0.20, 0.30, np.inf] (monthly fraction).
- 43. Implement a simplified Almgren-Chriss market impact cost: impact_cost = eta * (trade_size_USD / ADV_USD) ** 0.6, where eta=0.1 is a market impact coefficient and ADV_USD is the 30-day average dollar volume for each asset (fetch from yfinance).
- 44. For each combination (AUM, turnover_budget) in the grid: re-run the full 2019-2024 backtest applying both the turnover constraint in the optimizer and the market impact cost scaled by AUM. Record the resulting Sharpe ratio.
- 45. Store results in a 2D numpy array indexed by (AUM, turnover). Pass this to Max for visualization as the capacity frontier heatmap.
- 46. Identify and report the 'capacity breakpoint': the AUM level at which the Sharpe ratio drops by more than 25% relative to the \$1M baseline. This is the headline result of the project.

5. Deliverables Summary

Module	Owner	Deliverable	Timeline
M1 Data Pipeline	Moritz	data_loader.py with tests	Week 2

M2 Regime Classifier	Alessandro	regime_classifier.py + saved model	Week 2
M3 Signal Layer	Max	signals.py with test output	Week 4
M4 Optimizer	Moritz	optimizer.py with test block	Week 4
M5 Backtest	Alessandro	backtester.py + results CSV	Week 6
M5 Stress Test	Alessandro	stress_test.py + heatmap data	Week 8
Visualizations	Max	4 x PNG charts at 300 DPI	Week 10
GitHub Repository	All	README + clean notebooks	Week 10
Presentation	Max	10-slide deck (PowerPoint)	Week 11
Mini-Paper	All	3-4 page PDF	Week 12

6. 12-Week Timeline

Week	Alessandro	Moritz	Max
1-2	HMM prototype + feature engineering	Data pipeline (yfinance + FRED API)	Repo setup + README skeleton
3-4	Regime validation + visualization	cvxpy optimizer + constraints	Momentum signal + regime scaling
5-6	Walk-forward backtester v1	Integrate optimizer into backtest loop	Performance charts (3-strategy comparison)
7-8	Capacity frontier + stress test	Market impact model (Almgren-Chriss)	Heatmap visualization (Sharpe vs AUM)
9-10	Statistical validation of results	Code review + integration testing	Slide deck (10 slides)
11-12	Mini-paper methodology section	Mini-paper results section	Mini-paper layout + final polish

7. Module Interfaces and Integration Rules

Every module exposes a clean Python interface. No module should read raw data directly -- always import from the designated function. The following interface contract is binding: do not change function names or signatures without notifying the team.

Module	Function Signature	Returns
data_loader.py	get_prices(start, end)	DataFrame: dates x tickers
data_loader.py	get_features(start, end)	DataFrame: dates x macro features
regime_classifier.py	get_regime(date)	int: 0 (Risk-On), 1 (Risk-Off), 2 (Trans.)
signals.py	get_expected_returns(date, regime_label)	np.array of shape (8,): expected returns

optimizer.py	solve_portfolio(mu, Sigma, ...)	np.array of shape (8,): optimal weights
--------------	---------------------------------	--

Integration checkpoint at Week 6: all modules M1-M4 must be merged into a single notebook that runs end-to-end without errors. This checkpoint is non-negotiable -- if M4 is not ready, M5 cannot begin.

8. Repository Structure

The GitHub repository must follow this folder structure exactly. Each team member is responsible for their assigned files. Do not commit raw data files to the repository -- add a .gitignore that excludes CSV and pickle files.

Path	Description
README.md	Project overview, setup instructions, results summary
requirements.txt	All Python dependencies with version pins
src/data_loader.py	Module 1 (Moritz)
src/regime_classifier.py	Module 2 (Alessandro)
src/signals.py	Module 3 (Max)
src/optimizer.py	Module 4 (Moritz)
src/backtester.py	Module 5 Part A (Alessandro)
src/stress_test.py	Module 5 Part B (Alessandro)
notebooks/demo.ipynb	End-to-end demonstration notebook
results/figures/	All chart PNGs (Max)
results/metrics.csv	Summary performance table
models/hmm_model.pkl	Fitted HMM model file